

Deterministic Optimization

Discrete Optimization: Introduction

Shabbir Ahmed

Anderson-Interface Chair and Professor

School of Industrial and Systems Engineering

Discrete Optimization Challenges

Discrete Optimization: Challenges

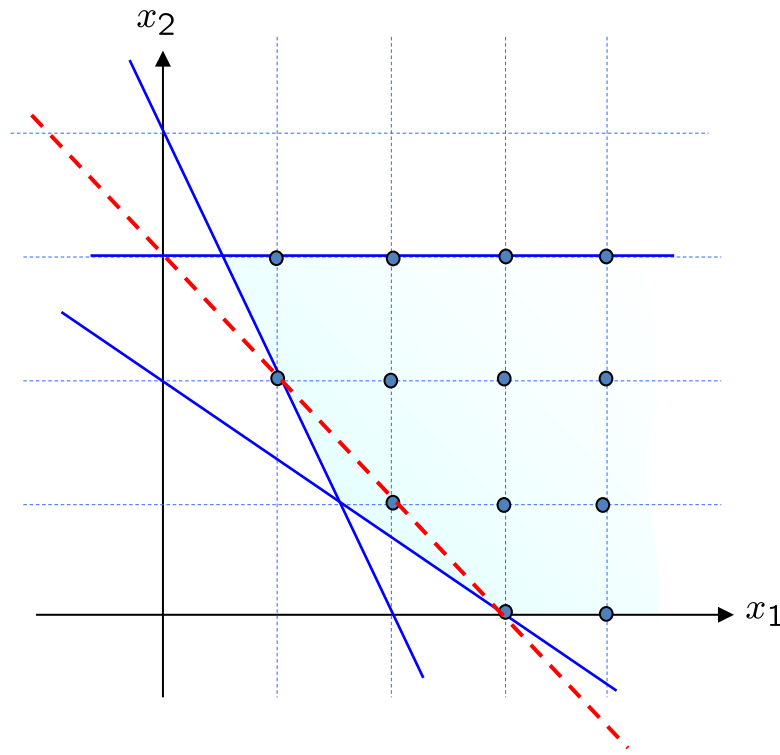
Learning Objectives

- Recognize difficulties of solving discrete optimization problems

Nonconvexity

- Discrete optimization involves optimization over a nonconvex set of feasible solutions

$$\begin{array}{ll}\min & x_1 + x_2 \\ \text{s.t.} & 2x_1 + x_2 \geq 4 \\ & x_1 + 1.5x_2 \geq 3 \\ & x_2 \leq 3 \\ & x_1, x_2 \geq 0 \text{ and integer}\end{array}$$



Local vs Global Optima

- Local optimal solutions may be very far from a global optimal solution
- Example:

$$\begin{array}{ll}\min & x + y \\ \text{s.t.} & 100x + y \geq 99 \\ & x, y \in \mathbb{Z}_+\end{array}$$

- The solution (1,0) is a global optimal solution with objective value 1
- The solution (0,99) is a “local” optimal solution with objective 99. It has equal or better objective than all “neighboring” feasible integer solutions.

Enumeration

- Small discrete optimization problems can be solved by enumerating all possibilities

- Example:

$$\begin{aligned} \max \quad & \sum_{j=1}^n r_j x_j \\ \text{s.t.} \quad & \sum_{j=1}^n c_j x_j \leq B \\ & x_j \in \{0, 1\} \quad \forall j = 1, \dots, n \end{aligned}$$

- There are 2^n possible values of the binary variables
- If n is small, we can check each to see if it is feasible, and then choose a feasible solution with maximum objective value

Enumeration in Python

```
1 # Solving an example discrete optimization problem by enumeration
2 import itertools
3 from random import randint
4 import time
5
6 # generate random problem instance
7 n = 25
8 r = [randint(0,9) for j in range(n)]
9 c = [randint(0,9) for j in range(n)]
10 B = sum(c)/2.0
11
12 # all possible solutions
13 solutions = list(itertools.product([0, 1], repeat=n))
14
15 xbest = []
16 vbest = -1
17
18 # check each solution
19 start_time = time.time()
20 for x in solutions:
21     # check feasibility
22     val = sum( [c[j]*x[j] for j in range(n)])
23     if val <= B:
24         obj = sum( [r[j]*x[j] for j in range(n)])
25         if obj > vbest:
26             vbest = obj
27             xbest = x
28
29 print 'n:', n
30 print 'Time:', (time.time() - start_time)
31
```

n	Time (secs)
5	0.0012
10	0.0049
15	0.1943
20	7.4745
25	311.9747

With n=50 estimated time > 330 years!!

Summary

- Discrete optimization problems are nonconvex
- Local solutions may be very far from global solutions
- Full enumeration can only work for very small problems